

Best Practices for System Prompts: A system prompt refers to the initial input or instruction that a model receives before generating text or responding. This prompt is crucial for the model’s operation [link](#).

Write Clear Instructions

- Why is it necessary to provide clear instructions to the model?

The model can’t read your mind. If the output is too long, you can ask the model to respond briefly. If the output is too simple, you can request expert-level writing. If you don’t like the format of the output, show the model the format you’d like to see. The less the model has to guess about your needs, the more likely you are to get satisfactory results.

Including More Details in Your Request Can Yield More Relevant Responses

To obtain highly relevant output, ensure that your input request includes all important details and context.

General Request	Better Request
How to add numbers in Excel?	How do I sum a row of numbers in an Excel table? I want to automatically sum each row in the entire table and place all the totals in the rightmost column named "Total."
Work report summary	Summarize my work records from 2023 in a paragraph of no more than 500 words. List the highlights of each month in sequence and provide a summary of the entire year.

Requesting the Model to Assume a Role Can Yield More Accurate Output

Add a specified role for the model to use in its response in the 'messages' field of the API request.

```
{
  "messages": [
    {"role": "system", "content": "You are Kimi, an artificial intelligence assistant."},
    {"role": "user", "content": "Hello, my name is Li Lei. What is 1+1?"}
  ]
}
```

Using Delimiters in Your Request to Clearly Distinguish Different Parts of the Input

For example, using triple quotes/XML tags/section headings as delimiters can help distinguish text parts that require different processing.

```
{
  "messages": [
    {"role": "system", "content": "You will receive two articles of the same length."},
    {"role": "user", "content": "<article>Insert article here</article><article>Insert article here</article>"}
  ]
}
```

```
{
  "messages": [
    {"role": "system", "content": "You will receive an abstract and the title of an article."},
    {"role": "user", "content": "Abstract: Insert abstract here.\n\nTitle: Insert title here."}
  ]
}
```

Clearly Define the Steps Needed to Complete the Task

It is advisable to outline a series of steps for the task. Writing these steps explicitly makes it easier for the model to follow and produces better output.

```
{
  "messages": [
    {"role": "system", "content": "Respond to user input using the following"},
    {"role": "user", "content": "\"\"\"Insert text here\"\"\""}
  ]
}
```

Provide Examples of Desired Output to the Model

Providing examples of general guidance is usually more efficient for the model's output than showing all permutations of the task. For instance, if you intend to have the model replicate a style that is difficult to describe explicitly in response to user queries, this is known as a "few-shot" prompt.

```
{
  "messages": [
    {"role": "system", "content": "Respond in a consistent style"},
    {"role": "user", "content": "Insert text here"}
  ]
}
```

Specify the Desired Length of the Model's Output

You can request the model to generate output of a specific target length. The target output length can be specified in terms of words, sentences, paragraphs, bullet points, etc. However, note that instructing the model to generate a specific number of words is not highly precise. The model is better at generating output of a specific number of paragraphs or bullet points.

```
{
  "messages": [
    {"role": "user", "content": "Summarize the text within the triple quotes"}
  ]
}
```

Provide Reference Text

Guide the Model to Use Reference Text to Answer Questions

If you can provide a model with credible information related to the current query, you can guide the model to use the provided information to answer the question.

```
{
  "messages": [
    {"role": "system", "content": "Answer the question using the provided art"},
    {"role": "user", "content": "<Insert article, each article enclosed in tr"}
  ]
}
```

Break Down Complex Tasks

Categorize to Identify Instructions Relevant to User Queries

For tasks that require a large set of independent instructions to handle different scenarios, categorizing the query type and using this categorization to clarify which instructions are needed may aid the output.

Based on the classification of the customer query, a set of more specific instructions can be provided to the model to help it handle subsequent steps. For example, assume the customer needs help with “troubleshooting.”

```
{  
  "messages": [  
    {"role": "system", "content": "You will receive a customer service inquiry"},  
  ]  
}
```

For Long-Running Dialog Applications, Summarize or Filter Previous Conversations

Since the model has a fixed context length, the conversation between the user and the model assistant cannot continue indefinitely.

One solution to this issue is to summarize the first few rounds of the conversation. Once the input size reaches a predetermined threshold, a query is triggered to summarize the previous part of the conversation, and the summary of the previous conversation can also be included as part of the system message. Alternatively, previous conversations throughout the entire chat process can be summarized asynchronously.

Chunk and Recursively Build a Complete Summary for Long Documents

To summarize the content of a book, we can use a series of queries to summarize each chapter of the document. Partial summaries can be aggregated and summarized to produce a summary of summaries. This process can be recursively repeated until the entire book is summarized. If understanding later parts requires reference to earlier chapters, then when summarizing a specific point in the book, include summaries of the chapters preceding that point.

Was this page helpful?

 Yes

 No